# Solitari: fra Monte Carlo, Poisson e AI

**Working paper / Nota didattica / versione 31 marzo 2026**

Sergio L Invernizzi

Società dei Naturalisti e Matematici di Modena, via Università 4, 41121 Modena
Centro Ricerche Didattiche U. Morin, via S. Giacomo 4, 31017 Pieve del Grappa (TV)
e–mail: `invernizzi2@gmail.com`

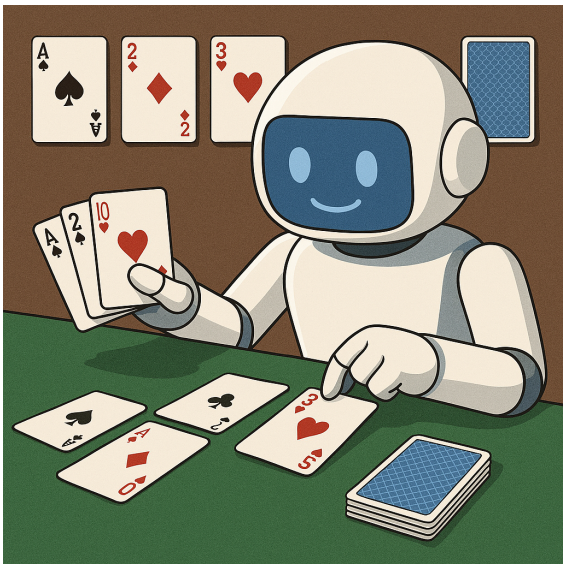

**Figura 1:** L'autore confessa di esser piuttosto preoccupato, ma nel contempo molto incuriosito, per le capacità delle AI di scrivere codici di programmazione estremamente efficaci per problemi di combinatorica non proprio banalissimi. Esse, le AI, vanno un po' incoraggiate, talvolta corrette ed aiutate dall'umano, ma l'umano da solo – credo – in certi casi non ce la farebbe.

Vediamo qui un piccolo "classico" dei solitari matematici, anche se non è molto noto al grande pubblico:

> *Da un mazzo di 40 carte (mischiato) coperte, si gira una carta alla volta; si vince se, in qualunque punto della sequenza, compaiono Asso, poi Due, poi Tre, in quest'ordine, di qualunque seme.*

Non ha un nome canonico,[1] perché è più un problema di probabilità che un vero solitario da tavolo; la condizione per vincere è puramente sequenziale: basta che da qualche parte nel mazzo ci sia una sottosequenza Asso-Due-Tre. Qui viene il bello: la probabilità di riuscita – per quanto

[1] Non va confuso con il *Solitario 1-2-3*, o "del carcerato", nel quale si gira il mazzo di 40 carte contando a voce "1, 2, 3, 1, 2, 3, ..." e si perde se la carta girata corrisponde al numero contato (Asso = 1, Due = 2, Tre = 3), ovvero si vince se si riesce a scartare tutto il mazzo. Per una discussione matematica di questo, si veda Barone, L., Un problema di calcolo delle probabilità: il solitario 1 2 3, *Archimede* 86 (1989), 57–62.

calcolabile formalmente – non è computazionalmente banale, ma si può stimare, e pure calcolare "esattamente". Vedremo quattro soluzioni diverse, concordanti.

**1. Simulazione Monte Carlo** La stima euristica più naturale è quella con un metodo Monte Carlo.[2] Qui codifichiano numericamente le carte: Asso → 0, Due → 1, Tre → 2, e scriviamo:

```
set.seed(123)
simula = function(nsim = 1e5) {
  x = rep(0:9, 4)
  conta = 0
for (i in seq_len(nsim)) {
    y = sample(x)
    if (any(
      y[1:(length(y)-2)] == 0 &
      y[2:(length(y)-1)] == 1 &
      y[3:length(y)] == 2)) {
      conta = conta + 1 }}
  conta/nsim }
simula(1e5)
```

```
[1] 0.041016
```

Quindi, con 100,000 (centimila) solitari simulati, abbiamo la stima $\mathbb{P}(\textit{successo}) \sim 0.041016$. Se ripetiamo la stima per 100 volte (per avere un'idea dell'errore) otteniamo il sommario (`summary( )`):

```
     Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
0.0392200 0.0401025 0.0406250 0.0406236 0.0410625 0.0422200
```

quindi $\mu = 0.0406236$, e deviazione standard (stima) $\sigma = 0.00067$. L'intervallo di confidenza la 95% (qualunque cosa significhi) riesce:

$$95\%\text{CI} \;\rightarrow\; \mu \mp 1.96\sigma/\sqrt{256} = [0.0405416,\;\; 0.0407056]$$

per cui possiamo assumere abbastanza "fiduciosamente" $\mathbb{P}(\textit{successo}) \sim 0.0406536$ come risultato del Monte-Carlo.

**2. Approssimazione di Poisson** La distribuzione modella, in opportune ipotesi, situazioni caratterizzate da eventi "rari".[3] Poisson pubblicò la formula applicandola inizialmente a contesti di probabilità giudiziaria. Gli eventi possono essere "rari" nel tempo,[4] oppure nello spazio, come nel caso del bombardamento di Londra con le V-1 tedesche, durante la Seconda Guerra Mondiale,

[2] Il metodo Monte Carlo è strettamente legato ai solitari. In una nota inedita del 1983, Stanislaw Ulam descrive come, nel 1946 durante la convalescenza da un'encefalite, giocando al *Canfield* (con 52 carte), si pose il problema delle probabilità di vittoria. Dopo tentativi combinatoriali falliti, ideò l'idea di simulare centinaia di partite per stimare empiricamente la probabilità, germe del metodo Monte Carlo, nome che richiamava appunto il casinò di Monte Carlo, dove lo zio di Stanislaw Ulam giocava spesso; il merodo fu sviluppato poi con John von Neumann, ancora impegnato a Los Alamos nel progetto della bomba ad idrogeno. Cf. Eckhardt, R., Stan Ulam, John von Neumann, and the Monte Carlo Method, *Los Alamos Science, Special Issue* 1987, 131–141; `https://mcnp.lanl.gov/pdf_files/Article_1987_LAS_Eckhardt_131--141.pdf`.

[3] Poisson, Siméon D. (1837). Probabilité des jugements en matière criminelle et en matière civile, précédées des règles générales du calcul des probabilités. Paris, France: Bachelier. Cf. Invernizzi, S., Rinaldi, M., & Comoglio, F. (2018). Moduli di matematica e statistica. Con l'uso di R (2ª ed.). Zanichelli.

[4] Nel 1898, Ladislaus Bortkiewicz rese popolare la distribuzione applicandola al numero di cavalieri prussiani morti per calci di cavallo.

analizzato da R. C. Clarke.[5] Clarke divise Londra in molte celle di area uguale e contò, per ogni cella, il numero di bombe cadute: la distribuzione del numero di bombe per cella era molto ben approssimata da una Poisson.[6] Infatti, gli impatti erano rari rispetto all'area totale, le cadute erano ('debolmente") indipendenti, ed il numero medio di bombe per cella era piccolo.

Formalizziamo allora il problema del solitario imitando il lavoro di Clarke:

- le celle di Londra $\iff$ le 38 triple consecutive nel mazzo;
- una bomba in una cella $\iff$ una comparsa di $(A, D, T)$ in quella tripla;
- il numero di bombe in una cella $\iff$ la variabile indicatrice $\mathbf{1}_{A_i}$.

Lo spazio campionario $\Omega$ è l'insieme di tutte le permutazioni $\omega$ delle 40 carte. Assumendo mescolamento uniforme, ogni permutazione ha probabilità 1/40!. Definiamo

$$N(\omega) = \sum_{i=1}^{38} \mathbf{1}_{A_i}, \text{ dove } A_i = \{\omega_i = A, \omega_{i+1} = D, \omega_{i+2} = T\}$$

Allora $N : \Omega \to \mathbb{N}$ è una v.a. discreta che misura quante volte compare la sottosequenza $(A, D, T)$ (in tre posiziomi consecutive). La $N = \sum_{i=1}^{38} \mathbf{1}_{A_i}$ del solitario è l'analogo al numero totale di bombe su Londra (in tutte le celle).

Cominciamo con la probabilità dell'evento che una tripla fissata, ad esempio la prima, sia $A, D, T$, che è uguale a:

$$p = E[\mathbf{1}_{A_1}] = \frac{4}{40} \cdot \frac{4}{39} \cdot \frac{4}{38} = \frac{4}{3705} \approx 0.0010796$$

in quanto $\frac{4}{40}$ è la probabilità che la prima carta su 40 sia uno dei quattro Assi, $\frac{4}{39}$ è la probabilità che la seconda carta sulle 39 rimaste sia uno dei quattro Due, e $\frac{4}{38}$ è la probabilità che la terza carta sia un Tre; le moltiplicazioni vengono dall'indipendenza.

Per la linearità del valore atteso (e per i conti già fatti)

$$\mathbb{E}[N] = \sum_{i=1}^{38} \mathbb{E}[\mathbf{1}_{A_i}] = 38 \times \frac{64}{40 \cdot 39 \cdot 38} = \frac{8}{195} \approx 0.041026$$

Dal punto di vista frequentista, in media, per ogni mescolamento, compaiono circa 0.041 sequenze $(A, D, T)$. Quindi con $\lambda = np = 38\,p = 38\,\frac{4}{3705} \approx 0.041026$ si ha

$$\mathbb{P}(N \geq 1) = 1 - \mathbb{P}(N = 0) = 1 - \frac{e^{-\lambda}\lambda^0}{0!} = 1 - e^{-\lambda} = 1 - e^{-8/195} \approx 0.040195$$

che è la probabilità teorica stimata con l'approssimazione di Poisson.

Qui abbiamo un *caveat* importante. Poniamo $A$=Asso, $D$=Due, $T$=Tre, $X$=qualunque. Entrambe le sequenze

$$s_1 = (\boxed{\text{A,D,T,}} X, X, \ldots), \quad s_2 = (X, \boxed{\text{A,D,T,}} X, X, \ldots)$$

fanno vincere e sono incompatibili; ma portano alla vittoria anche le

$$s_3 = (X, \boxed{A, D, T,} X, X, X, X, X, X, X, \ldots), \quad s_4 = (X, X, X, X, X, \boxed{A, D, T,} X, X, X, X, \ldots)$$

[5] Cf. Invernizzi *et al.; loc. cit.*

[6] Questo dimostrava secondo Clarke che le V-1 cadevano "a caso", quindi non secondo una pianificazione degli obiettivi dei nazisti: se ci fosse stata una tale pianificazionbe, l'approssimazione di Poisson non sarebbe stata buona come in effetti era.

che sono invece compatibili, se il mazzo è mescolato come

$$s = (X, \boxed{A, D, T,} X, \boxed{A, D, T,} X, X, X, \ldots)$$

Questo vuol dire che se semplicemente moltiplichiamo $p = 4/3705$ per $n = 38$, che sono le "finestre" ammissibili, otteniamo:

$$np = 38 \times p = 38 \times \frac{4}{3705} = \frac{8}{195} \approx 0.041026$$

che *deve* teoricamente essere *maggiore* della probabilità cercata,[7] in quanto fra i "casi favorevoli" la sequenza $s$ è contata (almeno) due volte: una come $s_3$ e (almeno) un'altra come $s_4$. In una presentazione didattica si potrebbe finire qui, osservando che le probabilità di sequenza come la $s$ sono dell'ordine di $p^2 \sim 10^{-6}$, e "quindi trascurabili".

Volendo proseguire, possiamo pensare intanto ad uno schema di tipo Poisson, ma è prudente allora

Per i dubbiosi ricordiamo ancora il Teorema di Le Cam (o Chen-Stein):[8] sia $\lambda = \mathbb{E}[N] = \sum_{i=1}^{n} p_i$. Allora $N$ può essere approssimata da Poisson($\lambda$) con errore piccolo se $\sum_{i<j} \mathbb{P}(X_i X_j) \ll \lambda$, in altre parole, la probabilità che due eventi accadano contemporaneamente deve essere molto più piccola della media totale, e questo accade nel solitario.

**3. Soluzione esatta da ChatGPT** Teoricamente è possibile evitare l'approssimazione con un calcolo esatto, assolutamente alla portata (non è un problema "impossibile"), ma praticamente, senza usare un calcolo assistito (programma, CAS, ecc.), la derivazione completa è lunga e piena di casi; molto più ragionevole quindi implementare un automa e far fare i conti a una macchina. Qui abbiamo usato ChatGPT (OpenAI, 2026) per un codice R:

```
options(digits=12)
memo <- new.env(hash = TRUE)
f <- function(a, d, t, x, s) {
  key <- paste(a, d, t, x, s, sep = "_")
  if (exists(key, envir = memo)) return(get(key, envir = memo))
  n <- a + d + t + x
  if (n == 0) { val <- 1; assign(key, val, envir = memo); return(val) }
  val <- 0
  if (a > 0) { sA <- 1; val <- val + (a / n) * f(a - 1, d, t, x, sA) }
  if (d > 0) { sD <- switch(as.character(s), "0"=0, "1"=2, "2"=0);
   val <- val + (d / n) * f(a, d - 1, t, x, sD) }
  if (t > 0) { if (s == 2) val <- val + 0 else { sT <- 0;
   val <- val + (t / n) * f(a, d, t - 1, x, sT) } }
  if (x > 0) { sX <- 0; val <- val + (x / n) * f(a, d, t, x - 1, sX) }
  assign(key, val, envir = memo); val
}
prob_lose <- f(4, 4, 4, 28, 0)
prob_win  <- 1 - prob_lose
prob_lose
prob_win
```

che ha dato l'output (con 12 cifre, per confronto):

[7] E questo è consolante in quanto la stima Monte Carlo è *minore* di $np$.

[8] Le Cam, L. (1960). An Approximation Theorem for the Poisson Binomial Distribution. *Pacific Journal of Mathematics* 10 (4): 1181–1197. In sintesi, il teorema stabilisce che se abbiamo una serie di eventi rari e *debolmente* dipendenti, il loro numero totale si comporta *quasi* come una variabile aleatoria di Poisson.

```
[1] 0.0406325583415
```

Si noti quanto sia umanamente criptico il codice, ed, in generale, quale enorme aiuto possa dare l'AI nella programmazione.[9]

**3. Soluzione esatta da Copilot** A Microsoft Copilot (Microsoft, versione disponibile nel febbraio 2026) abbiamo posto il problema in modo diretto e con un linguaggio maggiormente tecnico: "Sia B una sequenza random di 40 simboli ma dove esattamente 4 volte compare ciascuno dei 10 simboli 0,1,2,3,4,5,6,7,8,9. Calcolare la probabilità che una B casuale (con 4 copie di ogni simbolo) contenga almeno una volta 012." Ha risposto come segue (fornendo gentilmente il codice sorgente LaTeXqui usato):

---

**Problema:** $B$ è una sequenza casuale di lunghezza 40 contenente esattamente 4 copie di ciascuna cifra 0,1,...,9. Vogliamo calcolare la probabilità che $B$ contenga almeno una volta la sottosequenza consecutiva 012. Il numero totale di permutazioni del multinsieme è

$$T = \frac{40!}{(4!)^{10}}$$

**Idea chiave:** blocco 012 come simbolo unico.

Il pattern 012 non si sovrappone con sé stesso, quindi più occorrenze sono sempre disgiunte. Trattiamo ogni occorrenza di 012 come un blocco $X$: un blocco $X$ consuma una copia di 0, una di 1, una di 2 ; se ci sono $k$ blocchi 012, allora: restano $4-k$ zeri, $4-k$ uni, $4-k$ due; le cifre $3,\ldots,9$ restano con 4 copie ciascuna; e ci sono $k$ blocchi $X$. Il numero massimo di blocchi è $k_{\max} = 4$ (perché abbiamo solo 4 copie di ciascuna cifra). Dopo aver fuso $k$ blocchi, le molteplicità residue sono:

$$0^{4-k},\ 1^{4-k},\ 2^{4-k},\ 3^4,\ldots,9^4,\ X^k.$$

La lunghezza totale diventa:

$$n_k = (4-k)\cdot 3 + 7\cdot 4 + k = 40 - 2k.$$

Il numero di permutazioni con $k$ blocchi fusi è:

$$A_k = \frac{(40-2k)!}{(4-k)!^3\,(4!)^7\,k!}.$$

**Inclusione–esclusione.**[10] Il numero di sequenze che contengono almeno una volta 012 è:

---

[9] La AI, a richiesta, ha fornito anche varie versioni di "spiegazione di cosa fa il codice"; questa è quella maggiormente discorsiva:

La funzione $f$ modella un processo di estrazione casuale da un mazzo composto da quattro tipi di carte: $A, 2, 3$ ed $X$. A ogni chiamata, la funzione riceve il numero di carte rimanenti di ciascun tipo e uno stato interno $s$ che rappresenta il "suffisso" dell'eventuale sequenza $A$–2–3 già iniziata. L'obiettivo è calcolare la probabilità che, continuando a pescare fino all'esaurimento del mazzo, non venga mai completato il pattern proibito $A$–2–3. Per farlo, la funzione applica una ricorsione con memorizzazione: per ogni configurazione possibile del mazzo e dello stato $s$, memorizza il risultato così da non ricalcolare più volte la stessa situazione. A ogni passo, la funzione considera tutte le pescate possibili, pesandole con la loro probabilità (numero di carte di quel tipo diviso il totale delle carte rimanenti). Lo stato $s$ viene aggiornato in base alla carta pescata: un'$A$ avvia la sequenza, un 2 la prosegue solo se si era appena pescata un'$A$, un 3 completa il pattern se si era nella condizione "$A$ seguito da 2", mentre una $X$ interrompe sempre la sequenza. Se il mazzo si esaurisce senza aver mai completato $A$–2–3, la funzione restituisce 1; se invece la sequenza si completa in qualunque ramo della ricorsione, quel ramo contribuisce con probabilità 0. La probabilità finale è la somma pesata di tutti i rami "sicuri".

$$N_{\geq 1} = \sum_{k=1}^{4} (-1)^{k+1} A_k.$$

La probabilità cercata è quindi:

$$P = \frac{N_{\geq 1}}{T} = \sum_{k=1}^{4} (-1)^{k+1} \frac{(40-2k)!}{40!} \cdot \frac{(4!)^{10}}{(4-k)!^3 (4!)^7 k!}.$$

Semplificando:

$$P = \sum_{k=1}^{4} (-1)^{k+1} \frac{(40-2k)!}{40!} \cdot \frac{(4!)^3}{(4-k)!^3 k!}.$$

**Valore numerico approssimato:** $P \approx 0.0406$.

---

(fine della chat con Copilot) Abbiamo infine chiesto a *Mathematica Alpha* di calcolare $P$ in aritmetica esatta e di approssimare poi con 12 cifre, ottenendo

$$\sum_{k=1}^{4} (-1)^{k+1} \frac{(40-2k)!}{40!} \cdot \frac{(4!)^3}{(4-k)!^3 k!} = \frac{31248341}{769046850} \approx 0.0406325583415$$

**Conclusione** Abbiamo quindi alla fine questa tabellina di confronto per la probabilità di successo al solitario (fondamentalmente poco più del 4%):

| Metodo | $\mathbb{P}(\textit{successo})$ |
|---|---|
| Monte Carlo | 0.0406536 |
| Poisson($\lambda$) | 0.040195 |
| CharGPT | 0.0406325583415 |
| Copilot | 0.0406325583415 |

**Proposta didattica** Potrebbe essere interessante, ad esempio in una classe di 25, chiedere di giocare (a casa) al solitario, contando le prove (almeno 10 a testa) ed i successi. Per far prima non occorre girare le carte, basta mescolare bene e "dare le carte" una ad una.

---

Desidero ringraziare Stefano Horvath, per aver posto il problema, e Gabriella Caristi, per gli utili confronti tecnici sulle possibili soluzioni, e – *last-but-not-least* – chiunque avrà avuto tempo e voglia di leggere questa nota fino in fondo (e magari sperimentarla in aula).

[10] Per il principio di inclusione–esclusione (formule, dimostrazioni e relativa storia): `https://it.wikipedia.org/wiki/Principio_di_inclusione-esclusione`.